Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б. Н. Ельцина»
ИРИТ-РТФ

ЛАБОРАТОРНАЯ РАБОТА №4

Пирамидальная сортировка и очередь с приоритетом

ОТЧЕТ

Студент: Киприн Сергей Андреевич РИМ-250950

Екатеринбург 2026

Задачи

1. Реализовать алгоритм пирамидальной сортировки, попутно:
    a. Изучить такую структуру данных как Heap в вариациях Max Heap и Min Heap;
    b. Анализ сложности по времени и памяти в терминах O;
2. Решить задачу "Найти топ-k (сверху) элементов массива;
3. Решить задачу "Найти топ-k (снизу) элементов массива;
4. Используя полученные знания, реализовать структуру данных Priority Queue.

Ссылка на репозиторий Github

https://github.com/angst-storm/js-2026

Ход выполнения

Был реализован алгоритм пирамидальной сортировки на двоичной куче. Сначала из копии массива строится куча: для сортировки по возрастанию — Max Heap (в корне максимум), по убыванию — Min Heap. Затем корень (экстремум) меняется с последним элементом текущей кучи, размер кучи уменьшается на один, и через heapify свойство кучи восстанавливается. Так экстремумы по очереди «уезжают» в конец массива, и он оказывается отсортированным. Сложность: O(n log n) по времени, O(n) по памяти из-за копии входного массива.

```
    export function heapify(arr, size, root, ascending) {
      let extreme = root;
      const left = 2 * root + 1;
      const right = 2 * root + 2;

      if (ascending) {
        if (left < size && arr[left] > arr[extreme]) extreme =
left;
        if (right < size && arr[right] > arr[extreme]) extreme =
right;
      } else {
        if (left < size && arr[left] < arr[extreme]) extreme =
left;
        if (right < size && arr[right] < arr[extreme]) extreme =
right;
      }

      if (extreme !== root) {
        [arr[root], arr[extreme]] = [arr[extreme], arr[root]];
        heapify(arr, size, extreme, ascending);
      }
    }

    export function heapSort(arr, ascending = true) {
      const result = arr.slice();
      const n = result.length;

      for (let i = Math.floor(n / 2) - 1; i >= 0; i--) {
        heapify(result, n, i, ascending);
      }

      for (let i = n - 1; i > 0; i--) {
        [result[0], result[i]] = [result[i], result[0]];
        heapify(result, i, 0, ascending);
      }

      return result;
    }
```

Задачи топ-k сверху и снизу решены через кучу фиксированного размера k с переиспользованием heapify из пирамидальной сортировки. Для k наибольших строится Min Heap: в корне всегда самый слабый из текущих кандидатов, и, если встречается элемент больше корня — он его вытесняет. Для k наименьших симметрично используется Max Heap: в корне сильнейший из «дна», и его вытесняет элемент поменьше. Так не нужно сортировать весь массив: достаточно один раз пройти n элементов и поддерживать только окно из k лучших. Сложность: O(n log k) по времени, O(k) по памяти.

```
    import { heapify } from './heapsort.js';

    function topK(arr, k, largest) {
      if (k <= 0) return [];
      if (k >= arr.length) {
        return largest
          ? arr.slice().sort((a, b) => b - a)
          : arr.slice().sort((a, b) => a - b);
      }

      // сверху → Min Heap (ascending=false), снизу → Max Heap
(ascending=true)
      const ascending = !largest;
      const heap = arr.slice(0, k);

      for (let i = Math.floor(k / 2) - 1; i >= 0; i--) {
        heapify(heap, k, i, ascending);
      }

      for (let i = k; i < arr.length; i++) {
        const better = largest ? arr[i] > heap[0] : arr[i] <
heap[0];
        if (better) {
          heap[0] = arr[i];
          heapify(heap, k, 0, ascending);
        }
      }

      return largest
        ? heap.sort((a, b) => b - a)
        : heap.sort((a, b) => a - b);
    }

    /** Топ-k сверху: k наибольших элементов (по убыванию). */
    export function topKLargest(arr, k) {
      return topK(arr, k, true);
    }

    /** Топ-k снизу: k наименьших элементов (по возрастанию). */
    export function topKSmallest(arr, k) {
      return topK(arr, k, false);
    }
```

Очередь с приоритетом реализована на двоичной куче: по умолчанию Max Heap (извлекается наибольший элемент), опционально Min Heap. При enqueue элемент добавляется в конец и поднимается через siftUp, пока не восстановится свойство кучи; при dequeue из корня забирается приоритетный элемент, на его место ставится последний, и куча чинится вниз через heapify

из пирамидальной сортировки. peek просто смотрит корень без удаления. Сложность: enqueue/dequeue — O(log n), peek/size/isEmpty — O(1), память — O(n).

```
import { heapify } from './heapsort.js';

function siftUp(arr, index, ascending) {
  while (index > 0) {
    const parent = Math.floor((index - 1) / 2);
    const shouldSwap = ascending
      ? arr[index] > arr[parent]
      : arr[index] < arr[parent];

    if (!shouldSwap) break;

    [arr[index], arr[parent]] = [arr[parent], arr[index]];
    index = parent;
  }
}

export class PriorityQueue {
  constructor(maxHeap = true) {
    this.heap = [];
    this.ascending = maxHeap;
  }

  get size() {
    return this.heap.length;
  }

  isEmpty() {
    return this.heap.length === 0;
  }

  peek() {
    return this.isEmpty() ? undefined : this.heap[0];
  }

  enqueue(value) {
    this.heap.push(value);
    siftUp(this.heap, this.heap.length - 1, this.ascending);
  }

  dequeue() {
    if (this.isEmpty()) return undefined;
    if (this.heap.length === 1) return this.heap.pop();

    const top = this.heap[0];
    this.heap[0] = this.heap.pop();
    heapify(this.heap, this.heap.length, 0, this.ascending);
    return top;
  }
}
```

В index.js добавлены шаги для проверки работы алгоритмов.

```
29
30  // Пирамидальная сортировка O(n log n)
31  const heapSample = [3, 1, 4, 1, 5, 9, 2, 6];
32  console.log('\n4. Пирамидальная сортировка O(n log n)\n');
33  console.log(`вход:           [${heapSample.join(', ')}]`);
34  console.log(`по возрастанию: [${heapSort(heapSample).join(', ')}]`);
35  console.log(`по убыванию:    [${heapSort(heapSample, false).join(', ')}]`);
36
37  // Топ-k O(n log k)
38  const topKSample = [3, 1, 4, 1, 5, 9, 2, 6];
39  const k = 3;
40  console.log('\n5. Топ-k элементов O(n log k)\n');
41  console.log(`вход:  [${topKSample.join(', ')}], k=${k}`);
42  console.log(`сверху (k наибольших): [${topKLargest(topKSample, k).join(', ')}]`);
43  console.log(`снизу  (k наименьших): [${topKSmallest(topKSample, k).join(', ')}]`);
44
45  // Priority Queue O(log n) на операцию
46  console.log('\n6. Priority Queue\n');
47  const pq = new PriorityQueue(); // Max Heap
48  for (const value of [3, 1, 4, 1, 5, 9, 2, 6]) {
49    pq.enqueue(value);
50  }
51  console.log(`peek: ${pq.peek()}`);
52  const extracted = [];
53  while (!pq.isEmpty()) {
54    extracted.push(pq.dequeue());
55  }
56  console.log(`dequeue все элементы: [${extracted.join(', ')}]`);
```

Problems Output Debug Console Terminal Ports

```
4. Пирамидальная сортировка O(n log n)

вход:           [3, 1, 4, 1, 5, 9, 2, 6]
по возрастанию: [1, 1, 2, 3, 4, 5, 6, 9]
по убыванию:    [9, 6, 5, 4, 3, 2, 1, 1]

5. Топ-k элементов O(n log k)

вход:  [3, 1, 4, 1, 5, 9, 2, 6], k=3
сверху (k наибольших): [9, 6, 5]
снизу  (k наименьших): [1, 1, 2]

6. Priority Queue

peek: 9
dequeue все элементы: [9, 6, 5, 4, 3, 2, 1, 1]
angstorm@MacBook-M3-Pro---angstorm js-2025 %
```

Вывод

В ходе работы реализованы пирамидальная сортировка, поиск топ-k элементов и очередь с приоритетом на основе двоичной кучи с оценкой сложности по времени и памяти. Полученные решения показывают, как одна структура данных применяется к сортировке, выбору экстремумов и извлечению элементов по приоритету.